

Section 1: Welcome

Welcome to the course!

This first section contains a brief overview of the class. There are no lecture notes for this first section as it's an introduction to the rest of the class. This section is still important though, so make sure to watch the lecture videos to learn how to get the most out of the class.

Enjoy!

Section 2: Installing and Exploring Node.js

Lesson 1: Section Intro

In this section, you're going to set up your machine for the rest of the course. This includes installing Node.js and Visual Studio Code. This section also dives into what Node.js is, how Node.js works, and why Node.js is a tool worth learning.

Lesson 2: Installing Node.js and Visual Studio Code

In this lesson, you'll install Node.js and Visual Studio Code. Both are free, open source, and available for all operating system. They're the only tools needed to get started with Node!

Below are links to both tools. Take a moment to install them before continuing on with the class.

Links

- [Node.js](#)
- [Visual Studio Code](#)

Lesson 3: What is Node.js?

In this lesson, you'll explore what Node.js is. This includes a brief tour of the V8 JavaScript engine, non-blocking I/O, and more!

This lesson contains a presentation that covers what Node.js is. There are no notes for presentation lectures. Please refer to the video for details.

Lesson 4: Why Should I Use Node.js?

Why should you use Node.js? In this lesson, you'll learn what makes Node.js a tool worth using.

This lesson contains a presentation that covers the major advantages of Node.js. There are no notes for presentation lectures. Please refer to the video for details.

Lesson 5: Your First Node.js Script

It's time. In this lesson, you'll be creating and running your very first Node.js app.

Creating a Script

Node.js scripts are created with the `js` file extension. Remember that Node.js is not a programming language. All the code in this course is JavaScript code, which is why the `js` extension is used.

Below is an example script stored in a file named `index.js`.

```
console.log('Hello Node.js!')
```

Running a Script

You can run a Node.js script using the `node` command. Open up a new terminal window and navigate to the directory where the script lives. From the terminal, you can use the `node` command to provide the path to the script that should run. You can see an example of this command in the terminal below.

```
$ node index.js  
Hello Node.js!
```

When a Node.js script calls `console.log`, the logged values will show up in the terminal. This is a great way to get output from your Node.js application

Section 3: Node.js Module System

Lesson 1: Section Intro

The best way to get started with Node.js is to explore its module system. The module system lets you load external libraries into your application. That'll enable you to take advantage of built-in Node.js modules as well as third-party npm modules. This includes libraries for connecting to database, creating web servers, and more!

Lesson 2: Importing Node.js Core Modules

Node.js comes with dozens of built-in modules. These built-in modules, sometimes referred to as core modules, give you access to tools for working with the file system, making http requests, creating web servers, and more! In this lesson, you'll learn how to load in those core modules and use them in your code.

Importing Node.js Core Modules

To get started, let's work with some built-in Node.js modules. These are modules that come with Node, so there's no need to install them.

The module system is built around the `require` function. This function is used to load in a module and get access to its contents. `require` is a global variable provided to all your Node.js scripts, so you can use it anywhere you like!

Let's look at an example.

```
const fs = require('fs')  
fs.writeFileSync('notes.txt', 'I live in Philadelphia')
```

The script above uses `require` to load in the `fs` module. This is a built-in Node.js module that provides functions you can use to manipulate the file system. The script uses `writeFileSync` to write a message to `notes.txt`.

After you run the script, you'll notice a new `notes.txt` file in your directory. Open it up and you'll see, "I live in Philadelphia!".

Links

- [Node.js documentation](#)

- [Node.js fs documentation](#)

Lesson 3: Importing Your Own Files

Putting all your code in a single file makes it easy to get started with Node.js. As you add more code, you'll want to stay organized and break your Node.js app into multiple scripts that all work together. In this lesson, you'll learn how to create a Node.js application that's spread out across multiple files.

Importing Your Own Files

You know how to use `require` to load in built-in modules. `require` can also be used to load in JavaScript files you've created. All you need to do is provide `require` with a relative path to the script you want to load. This path should start with `./` and then link to the file that needs to be loaded in.

```
const checkUtils = require('./src/utils.js')

checkUtils()
```

The code above uses `require` to load in a file called `utils.js` in the `src` directory. It stores the module contents in a variable, and then uses the contents in the script.

Exporting from Files

Node.js runs the scripts that you require. That means the `require` call above will cause `utils.js` to run. Node.js provides the required script with a place to store values that should be exported as part of the library. This is on `module.exports`.

You can see `utils.js` below. A function is defined and then assigned to `module.exports`. The value stored on `module.exports` will be the return value for `require` when the script is imported. That means other scripts could load in the utilities to access the `check` function.

```
const check = function () {
  console.log('Doing some work...')
}

module.exports = check
```

If you run the original script, you'll see the message that logged from the `check` function in `utils.js`.